# Requirement-Aware Context Compression for Simulation-in-the-Loop Agents

Tazeem Mahashin[1] Ian Sun[2] Angela Choi[3] Matthew Han[4]

[1]Rensselaer Polytechnic Institute [2]Massachusetts Institute of Technology
[3]University of Pennsylvania [4]Georgia Institute of Technology

## Abstract

Agentic loops that call a numerical simulator re-send the simulator's output to the language model after every revision. For transient physical simulations this output is a dense, multi-variable time series—hundreds of kilobytes of numeric CSV per iteration—and it dominates the token cost of the loop because it *recurs* at every step. General-purpose text compressors barely help here: they remove *linguistic* redundancy, of which dense numeric output has almost none. We argue that the right axis to compress along is not redundancy but *irrelevance*, and that what is relevant is defined by the downstream task's own machine-checkable requirements. We present *requirement-aware context compression*: the agent runs the simulation at full fidelity, evaluates it deterministically against the task's pass/fail predicates, and forwards only a requirement-keyed verdict—one line per requirement, with the exact offending value for any failure. On a real rocket-engine design loop of 14 iterations, this compresses 2,379,129 tokens of raw simulator output to 3,065 (a 99.87% reduction) while preserving the decision-relevant signal byte-exact. Measured in a byte-level compressor's own tokenizer, our pass takes a single iteration from 198,358 to 433 tokens (99.78%), where that compressor alone achieves 0.45% and an LLM summary achieves 99.02% but *drops* the failing value the design loop needs. The technique is complementary to byte-level compression—which further shrinks our verdict by 27%—and is domain-agnostic: it applies to any requirement-checked time-series system.

## 1 Introduction

A growing class of agents close a loop around a non-LLM tool: the model proposes an artifact, a deterministic engine evaluates it, and the model revises. When the engine is a physical simulator, the evaluation it returns is not a short status code but a full *transient time series*—every state variable of every component at every timestep. In the design loop we study (Section 3), a single simulation emits on the order of $1.7 \times 10^5$ tokens of numeric CSV. A naive agent forwards that output back to the model so it can revise; because the loop iterates, this cost is paid *again on every iteration*, and it is by a wide margin the dominant term in the loop's token budget.

The obvious mitigation is to compress the tool output before sending it. But the mainstream of context compression targets the wrong quantity. Prompt-compression methods such as LLMLingua [1, 2] and learned byte-level compressors exploit the fact that natural language is statistically redundant: many tokens are predictable from their neighbors and can be dropped or merged with little information loss. Dense numerical output has almost none of this redundancy—each value is close to incompressible—so a general compressor correctly leaves it nearly untouched. We confirm this directly: a state-of-the-art hosted byte-level compressor reduces our raw simulator output by only 0.45% (Section 6).

Our central observation is that the simulator output is nonetheless *enormously* compressible, just not along the redundancy axis. It is compressible along the axis of *relevance*: of the hundreds of thousands of numbers in the time series, the next design revision depends on only a handful—precisely those that the task's acceptance criteria test. Crucially, in a requirement-driven loop those criteria are already written down, as machine-checkable predicates over the simulator's output fields. We can therefore condition the compressor on them.

We call this *requirement-aware context compression*. Rather than paraphrase the raw output, the agent (i) runs the simulation at full fidelity, (ii) evaluates the task's predicates against the complete time series in deterministic code, and (iii) forwards only a *requirement-keyed verdict*: one line per requirement, marked pass or fail, annotated for each failure with the exact actual value, the statistic and field it came from, and the threshold it violated. Everything that does not bear on a requirement is dropped. What survives is exactly the signal the model needs to revise, and its size is $O(\#\text{requirements})$—independent of the length of the time series.

**Contributions.**

- We identify *relevance*, not redundancy, as the compressible axis of numeric tool output in requirement-driven agent loops, and formalize a compression operator conditioned on the task's machine-checkable requirements (Section 4).
- We give a domain-agnostic kernel, `compress_tabular_context`, that applies to any list-of-rows checked against `(field, stat, op, value)` predicates—logs versus SLOs, metrics versus thresholds, benchmark output versus targets—and a `protect()` discipline that lets the verdict be safely stacked under a lossy byte-level compressor while keeping every critical number byte-exact.
- On a real 14-iteration rocket-engine design loop we measure a 99.87% reduction (2,379,129 → 3,065 tokens). In a head-to-head using a byte-level compressor's own tokenizer, our verdict reaches 99.78% (198,358 → 433) where the byte-level compressor reaches 0.45% and an LLM summary reaches 99.02% but discards the decision-relevant value (Section 6).
- We show the two approaches are *complementary*: a byte-level compressor shaves a further 27% off our already-tiny verdict, so the right architecture applies requirement-aware compression to the heavy numeric tool-output and byte-level compression to the static prose prefix.

## 2 Related Work

**Prompt and context compression.** LLMLingua and LongLLMLingua [1, 2] compress prompts by deleting low-information tokens under a small language model, exploiting the redundancy of natural language; Selective Context [3] prunes by self-information. These operate on prose and are task-agnostic—they do not know what the reader will do with the text. Our method is orthogonal: it compresses a structured numeric artifact by conditioning on the consumer's explicit acceptance test, and it composes with a byte-level prose compressor rather than competing with it (Section 6).

**Retrieval and summarization.** Retrieval-augmented generation [4] and abstractive summarization reduce what reaches the model by selecting or rewriting passages. Both are lossy in an uncontrolled way for our setting: a summarizer asked to condense a dense numeric table has no principled basis for which of the thousands of values to keep, and—as we measure in Section 6—spends its budget on salient-looking prose while silently dropping the one value the loop tests against. Requirement-aware compression makes that selection aligned with the downstream check *by construction*.

**Tool-using and self-refining agents.** ReAct [5], Reflexion [6], and Toolformer [7] interleave model reasoning with external tool calls and feed tool observations back into context. Our work targets the cost of that feedback channel when the tool is a high-volume numerical simulator, and replaces the raw observation with a task-conditioned verdict. The idea of returning a structured judgement rather than raw output is related to verifier- and checker-in-the-loop training [8], but we apply it as an *inference-time context-compression* mechanism.

## 3 Problem Setting

**The loop.** We study an LLM-in-the-loop design system for rocket propellant feed systems. The loop is

$$\text{spec} \to \underbrace{\text{LLM design}}_{\text{revise}} \to \text{sim} \to \text{verdict} \to \cdots$$

A *spec* fixes the design target and carries a set of machine-checkable *checks*. The model emits a network of nodes (tanks, engine) and connections (feed lines with effective areas), the simulator integrates the transient fluid network, and the resulting state is evaluated against the checks to decide whether to stop or revise.

**What the simulator emits.** The simulator produces two CSV tables per run, `nodes.csv` and `connections.csv`, holding every node/connection × every field (pressure, temperature, density, internal energy, mass flow, mixture ratio, thrust, ...) × every timestep. Runs reach tens of thousands of rows and hundreds of kilobytes. At a conservative 4 characters/-token this is $\approx 1.7 \times 10^5$ tokens per iteration; measured in a production byte-level tokenizer the same output is $\approx 1.98 \times 10^5$ tokens, because dense numerics tokenize *more* finely than prose (Section 6).

**Cost model.** Let $R$ be the per-iteration size of the raw simulator output and $V$ the size of the compressed verdict, with $V \ll R$. A loop that forwards raw output pays $\Theta(R)$ of *recurring* input per iteration on top of the conversation; over $N$ iterations the cumulative simulator-output cost is $\Theta(NR)$, and if the transcript is replayed it grows as $\Theta(N^2R)$. Requirement-aware compression replaces $R$ with $V$, making the recurring term $\Theta(V)$ with $V = O(\#\text{requirements})$, independent of the time-series length. Because $R$ is nearly three orders of magnitude larger than $V$ ($R/V \approx 780$ here), this term effectively disappears (Figure 1).

# 4 Method

## 4.1 Requirement-aware compression operator

Let the simulation produce a record sequence $X = (x_1, \ldots, x_T)$, where each $x_t$ maps field names to values at timestep $t$. Let the task carry requirements $Q = \{q_1, \ldots, q_m\}$, each a predicate

$$q_i = \big(f_i,\ s_i,\ \triangleright_i,\ \theta_i,\ d_i\big),$$

with field $f_i$, statistic $s_i \in \{\mathsf{final}, \mathsf{first}, \mathsf{min}, \mathsf{max}, \mathsf{mean}, \mathsf{count}\}$, comparison operator $\triangleright_i \in \{\geq, \leq, >, <, =, \neq\}$, threshold $\theta_i$, and human description $d_i$. Define the column aggregator

$$A(X, f, s) = s\big(\langle x_t[f] \rangle_{t:\, x_t[f] \neq \perp}\big),$$

i.e. apply statistic $s$ to the non-missing values of field $f$ across all timesteps. The compression operator is

$$\mathcal{C}(X, Q) = \Big\langle \big(\mathbb{1}[a_i \triangleright_i \theta_i],\ d_i,\ \rho_i\big) \Big\rangle_{i=1}^{m},\ a_i = A(X, f_i, s_i),$$

where the per-requirement payload is

$$\rho_i = \begin{cases} \varnothing & \text{if } a_i \triangleright_i \theta_i \text{ (pass)}, \\ \big(s_i(f_i),\ \triangleright_i,\ \theta_i,\ a_i\big) & \text{otherwise (fail)}. \end{cases}$$

Passing requirements contribute only a tag; failing ones additionally carry the exact actual value $a_i$ and the violated predicate, so the model sees precisely *what* failed and *by how much*. The serialized verdict has length $O(m)$ and is independent of $T$. An example produced by the real loop is shown in Listing 1.

**Listing 1:** A real verdict for one iteration of the `lox_methane_engine` spec (8 checks). The raw simulator output for this same iteration is ≈198k tokens; this verdict is 433.

```
VERDICT: 5/8 checks passed

[PASS] ran: Simulation runs without crash.
[PASS] flow_happens: Propellant actually flows.
[FAIL] thrust_min: End-of-burn thrust >= 1500 N.
   expected >= 1500.0; actual=1136.79
   (engine.thrust.final=1136.79)
[PASS] thrust_max: End-of-burn thrust <= 2500 N.
[PASS] mr_min: Mixture ratio >= 2.8.
[PASS] mr_max: Mixture ratio <= 3.6.
[FAIL] chamber_pressure: P_c >= 1.2 MPa.
   expected >= 1200000.0; actual=970428
[FAIL] isp: Specific impulse >= 195 s.
   expected >= 195.0; actual=193.132

TARGETED REVISION ADVICE:
 - Thrust low (1137 N vs 1500). Increase ox
   and fuel CdA together by ~1.52x.
```

## 4.2 Why the verdict preserves the signal

The verdict is lossy with respect to the raw time series but *lossless with respect to the decision*: by construction it contains every quantity any requirement tests, at full precision, plus the sign and magnitude of each violation. The model never has to parse a $10^5$-token table to recover the handful of numbers that matter; it receives them directly, already attributed to the check they break. This is why the compressed representation is not merely smaller but a *better* input: it removes noise, not signal (Section 6).

## 4.3 Domain-agnostic kernel

Nothing above is rocket-specific. The same operator applies to any list-of-rows checked against requirements. We expose it as `compress_tabular_context(records, requirements)`, taking generic rows and a requirement set of `(field, stat, op, value)` predicates and emitting the same requirement-keyed verdict. A 5,000-row request-latency log checked against a single SLO ($\mathsf{max}$(`latency_ms`) $< 500$) compresses to a one-line verdict with the offending maximum—>95% reduction versus serializing the log—by exactly the mechanism used for the simulator.

## 4.4 Composition with byte-level compression

Requirement-aware compression and byte-level prose compression are orthogonal and compose. We apply the former to the heavy numeric tool-output and the latter to the static prose prefix (system prompt, spec, seed design). Because a learned byte-level compressor is lossy, we wrap every critical span—spec JSON, seed design, and the verdict's numerals—in a `protect()` fence that instructs the compressor to pass those bytes through verbatim. This lets us stack a hosted compressor on top of the verdict without risking corruption of a value the loop depends on. Empirically the byte-level pass removes a further 27% of the already-tiny verdict (Section 6).

# 5 Experimental Setup

**Workload.** We measure on three real multi-iteration design runs from the rocketcursor loop: `lox_methane_engine` (a LOX/methane engine, 8 checks, 6 iterations), `nitrogen_blowdown_vent` (a blowdown vent, 2 iterations), and `pressure_window_blowdown` (9 checks, 6 iterations)—14 iterations in total. For each iteration we have the simulator's raw `nodes.csv` and `connections.csv` and the recorded result the verdict is computed from.

**What we compare.** For every iteration we compare the bytes a naive loop would forward—the raw CSV concatenation—against the verdict our loop actually sends. The savings are *recurring*: the raw cost is paid every iteration of a multi-step loop.

**Table 1:** Requirement-aware reduction over 14 real iterations (4-char/token estimate). *sent* is the verdict actually forwarded to the model.

| spec | iters | raw tok | sent tok | reduction |
|---|---|---|---|---|
| `lox_methane_engine` | 6 | 334,882 | 1,491 | 99.55% |
| `nitrogen_blowdown_vent` | 2 | 1,286,112 | 226 | 99.98% |
| `pressure_window_blowdown` | 6 | 758,135 | 1,348 | 99.82% |
| **Total** | **14** | **2,379,129** | **3,065** | **99.87%** |

**Tokenization.** Two regimes appear, and we keep them distinct. (i) For the aggregate sweep we use a tokenizer-agnostic estimate of 4 characters/token; the compression *ratio* is robust to the exact tokenizer. (ii) For the head-to-head against a byte-level compressor we report that compressor's *own* tokenizer counts, which is the apples-to-apples basis for comparing against it—and which counts the raw output *higher* ($1.98 \times 10^5$) than the 4-char estimate ($1.70 \times 10^5$), because dense numerics tokenize finely. Our reported head-to-head reduction is thus if anything conservative.

**Baselines.** We compare against (a) a hosted byte-level compressor (`bear-2`) run on the raw output at low and maximum aggressiveness, and (b) an LLM summarization baseline: a frontier model (`claude-sonnet-4-6`) prompted to summarize the raw CSV for the design model while "preserving every engineering quantity that could matter for a pass/fail decision." All numbers are produced by `python -m loop.compression_benchmark` and the accompanying scripts; raw outputs and the saved `benchmark.json` accompany the code.

# 6 Results

## 6.1 Aggregate reduction

Table 1 reports the per-run and total reduction across all 14 iterations. The loop's 2,379,129 tokens of raw simulator output compress to 3,065—a 99.87% reduction, averaging $\approx 1.7 \times 10^5$ raw down to $\approx 220$ sent per iteration.

## 6.2 Head-to-head vs. byte-level and summarization

Table 2 compares, on a single iteration and in the byte-level compressor's own tokenizer, four ways of reducing the raw output. The general byte-level compressor is near-inert on dense numerics (0.45% at maximum aggressiveness): there is little linguistic redundancy to exploit. The LLM summarization baseline does shrink the output (99.02%), but it is both 4.5× larger than our verdict *and* lossy on the decision: in our run it spent its budget describing tank states and never emitted the end-of-burn engine thrust (1136.79 N) that the spec's binding `thrust_min` check tests—the single most decision-relevant number is simply absent. Requirement-aware compression reaches 99.78% (458×) and preserves that value byte-exact.

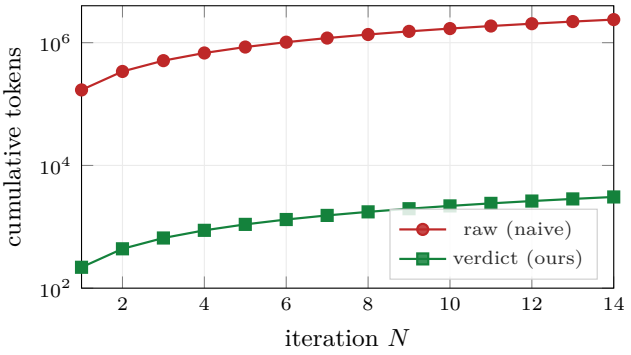


**Figure 1:** Cumulative simulator-feedback tokens over the 14-iteration workload (log scale). The recurring raw-output term ($\approx 1.7 \times 10^5$/iter) is nearly three orders of magnitude larger than the verdict ($\approx 220$/iter).

**Table 2:** One iteration of `lox_methane_engine`, measured in the byte-level compressor's own tokenizer. "keeps actual?" is whether the decision-relevant failing value (1136.79 N) survives.

| method | raw→out | reduction | keeps actual? |
|---|---|---|---|
| **Ours (req.-aware)** | 198,358→**433** | **99.78%** | yes |
| `bear-2`, aggr. 0.5 | 198,358→198,356 | 0.00% | — |
| `bear-2`, aggr. 0.9 | 198,358→197,466 | 0.45% | — |
| LLM summary | 198,358→1,948 | 99.02% | no |

## 6.3 The methods compose

Requirement-aware compression and byte-level compression remove different things, so they stack. Running the hosted `bear-2` compressor *on our verdict* shrinks it a further 27% (433 → 315 tokens). The intended architecture therefore uses requirement-aware compression for the heavy numeric tool-output and byte-level compression for the static prose prefix, with `protect()` fences (Section 4.4) keeping every critical numeral byte-exact through the lossy pass.

## 6.4 Quality of the compressed signal

The compression preserves the loop's ability to revise. Acting only on the ≈ 220-token verdict—never on the raw time series—the model makes targeted, attributable design changes: on `lox_methane_engine` it improves from 5/8 to 6/8 passing checks while adjusting exactly the feed areas the verdict's thrust failure points to (Listing 1). The verdict carries the failing magnitude and a derived revision direction, so the model is never asked to recover a control signal by parsing $10^5$ tokens of CSV. Consistent with the summarization result above, this

supports the claim that the verdict is a *better* input than the raw output, not merely a cheaper one. We do not claim the loop reaches all-pass on every spec within the recorded budget; our claim is on the fidelity and cost of the feedback channel, which the data above establish.

# 7 Discussion

**Why requirement-awareness is the right axis.** The three baselines in Table 2 cleanly separate two compressible axes. Byte-level compression targets redundancy and is near-inert here because numeric output has none. Summarization targets salience but has no principled link to the downstream decision, so it discards the tested value. Requirement-aware compression targets relevance and grounds it in the task's own predicates, which both bounds the output size by the number of requirements and guarantees the tested quantities survive. The win is not a better squeeze of the same information; it is compressing a different, smaller object—the decision—rather than the data.

**Generality.** The operator needs only (i) a high-volume tabular/time-series tool output and (ii) machine-checkable requirements over its fields. Both are common far outside rocketry: service logs against SLOs, monitoring metrics against alert thresholds, financial series against risk limits, CI/benchmark output against regression targets. In each case the same `(field, stat, op, value)` kernel applies.

**Limitations.** The verdict carries only what the requirements ask for; a quantity that is decision-relevant but *unchecked* will be dropped, so the method is only as complete as the requirement set. This is a feature for cost and a risk for discovery: exploratory analysis that needs the full series should bypass the compressor. The aggregator currently supports scalar statistics over fields; requirements that depend on temporal *shape* (e.g. overshoot duration) require richer statistics, which the same framework admits as additional `stat` kinds. Finally, our quality evidence is the loop's targeted revision behavior and a single measured summarization comparison; a larger controlled study of downstream solve rate under matched budgets is future work.

# 8 Conclusion

Numeric tool output in agent loops is barely compressible along the redundancy axis that general compressors exploit, yet it is extraordinarily compressible along the axis of relevance—and in requirement-driven loops the relevant quantities are already specified as machine-checkable predicates. Conditioning the compressor on those predicates turns a recurring $10^5$-token feedback channel into a $\sim 10^2$-token verdict that is lossless with respect to the decision, preserves critical values byte-exact, and composes with byte-level compression of the prose prefix. Across a real 14-iteration design loop this is a 99.87% reduction with no loss of—indeed an improvement in—the signal the model revises against. The mechanism is domain-agnostic, and we expect it to apply wherever an agent loops around a high-volume, requirement-checked tool.

**Reproducibility.** The compressor (`loop/compression.py`), the deterministic evaluator (`loop/evaluator.py`), the benchmark (`loop/compression_benchmark.py`, including the `-ttc` head-to-head), and the saved results (`results/compression/benchmark.json`) accompany this paper. The aggregate table reproduces with `python -m loop.compression_benchmark`; the head-to-head and summarization baselines require API keys for the respective hosted services.

# References


[1] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. *EMNLP*, 2023.

[2] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. *ACL*, 2024.

[3] Y. Li, B. Dong, F. Guerin, and C. Lin. Compressing Context to Enhance Inference Efficiency of Large Language Models. *EMNLP*, 2023.

[4] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.

[6] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS*, 2023.

[7] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS*, 2023.

[8] K. Cobbe et al. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*, 2021.